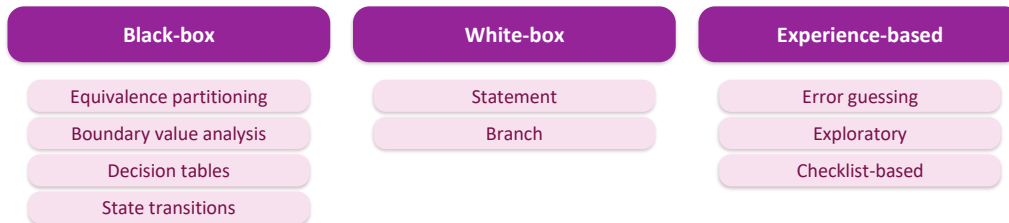


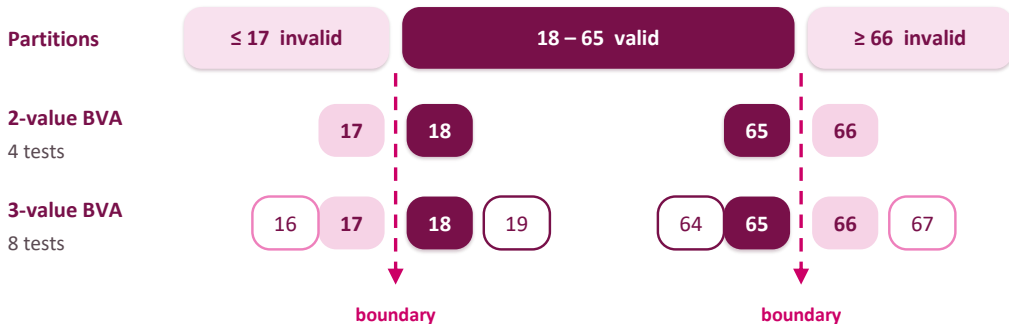
4. Test Analysis and Design

A. Test techniques overview



- Black-box:** from specified behaviour; tests stay valid if the implementation changes
- White-box:** from internal structure; only after design/implementation
- Experience-based:** tester knowledge; complements the others

B. Boundary value analysis (BVA)



Used when partitions are **ordered** (e.g., ages). The **boundary values** are the smallest and largest values in each partition; defects often sit at these edges.

- 2-value BVA:** test each boundary value and its nearest neighbour in the next partition (e.g., 17 and 18)
- 3-value BVA:** test each boundary value and the values just either side of it (e.g., 17, 18 and 19); more thorough

If every boundary value (and neighbour) has a test, that is 100% BVA coverage.

Exam tip: 3-value BVA catches “if ($x \leq 10$)” coded as “if ($x = 10$)” via $x = 9$; 2-value doesn’t.

C. Equivalence partitioning (EP)



Divide the possible values into **equivalence partitions**: groups of values the system should treat the same way. One test value per partition is enough, because every other value in it should behave the same.

- Partitions must **not overlap** and must **not be empty**
- Valid** partitions hold values the system should accept; **invalid** partitions hold values it should reject

Coverage items are the partitions, including invalid ones. **If every partition has at least one test, that is 100% partition coverage.** Coverage = partitions tested $\div$ total partitions $\times$ 100%.

With several inputs, **Each Choice coverage** means every partition of every input appears in at least one test.

D. Decision table testing

Conditions / actions	R1	R2	R3	R4
Card valid?	F	T	T	T
PIN correct?	—	F	T	T
Sufficient funds?	—	—	F	T
Reject card	X			
‘Incorrect PIN’		X		
‘Insufficient funds’			X	
Dispense cash				X

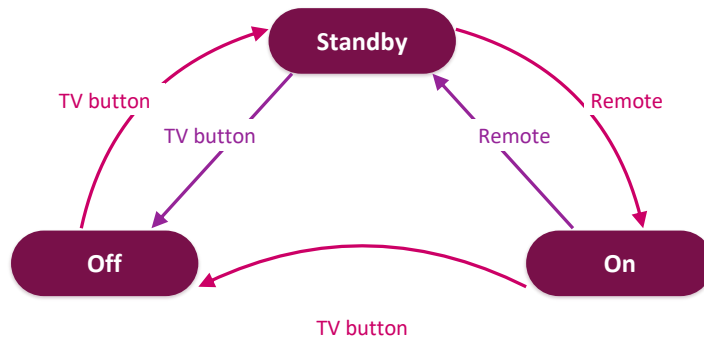
Use when **combinations of conditions** lead to different actions. Each **column** is one rule: a combination of conditions and the actions that result.

T / F = condition true / false · **—** = doesn’t matter · **N/A** = impossible combination · **X** = the action happens.

Coverage items are the possible columns. If every column has a test, that is 100% decision table coverage.

Good for finding gaps and contradictions in requirements. Tables grow quickly as conditions are added, so merge columns that lead to the same result (minimise) or focus on risk.

E. State transition testing



Models how a system moves between **states** when **events** happen. A transition is written **event [guard] / action**: the guard is a condition that must be true, the action is what the system does.

A **state table** lists every state–event pair, so it also shows **invalid** transitions.

- **All states coverage**: every state is visited (weakest)
- **Valid transitions coverage** (0-switch): every valid transition is exercised (most common)
- **All transitions coverage**: every valid transition is exercised and every invalid one is tried; the minimum for safety-critical software

Try only one invalid transition per test case, so one defect can't hide another (**defect masking**).

F. Experience-based techniques



- **Error guessing**: anticipate errors, defects and failures from experience; **fault attacks** use a list of them
- **Exploratory testing**: design, execute and learn at once; useful with poor specs or time pressure
- **Checklist-based**: cover checklist items (often questions); keep lists short and updated

G. ATDD



- Tests written **before** implementation, by customers, developers and testers
- Order: **positive**, then **negative**, then **non-functional**
- Cover the whole story, but don't go beyond it

H. Collaborative user stories

US-88 Apply a discount code

As a shopper,
I want to apply a discount code
so that I pay less.

Acceptance criteria

- valid code → 10% off
- expired code → error

Card

the medium

Conversation

how it's used

Confirmation

acceptance criteria

User stories, acceptance criteria and ATDD are **collaboration-based test approaches**: as well as helping detect defects, they **also** focus on **avoiding** defects through collaboration and communication.

Good stories are **INVEST**: Independent, Negotiable, Valuable, Estimable, Small, Testable.

I. Statement and branch testing

- **Statement testing**: the coverage items are the executable statements. 100% means every statement ran at least once, but defects that depend on particular data values can still be missed
- **Branch testing**: the coverage items are branches, every possible jump between statements, including both outcomes of each decision. 100% means every branch was taken, but defects that only appear on one specific path can still be missed

Exam tip: 100% **branch** coverage guarantees 100% **statement** coverage, not the other way round.

J. Value of white-box testing

- Uses the whole implementation, so it works even with vague or outdated specs
- Can miss **defects of omission** (unimplemented requirements)
- Usable statically (dry runs, pseudocode)
- Gives an **objective coverage measure**

K. Acceptance criteria

Conditions for stakeholder acceptance: effectively the **test conditions**. Used to define scope, reach consensus, describe positive and negative scenarios, and plan and estimate.

- **Scenario-oriented**: Given / When / Then
- **Rule-oriented**: verification list or input–output table